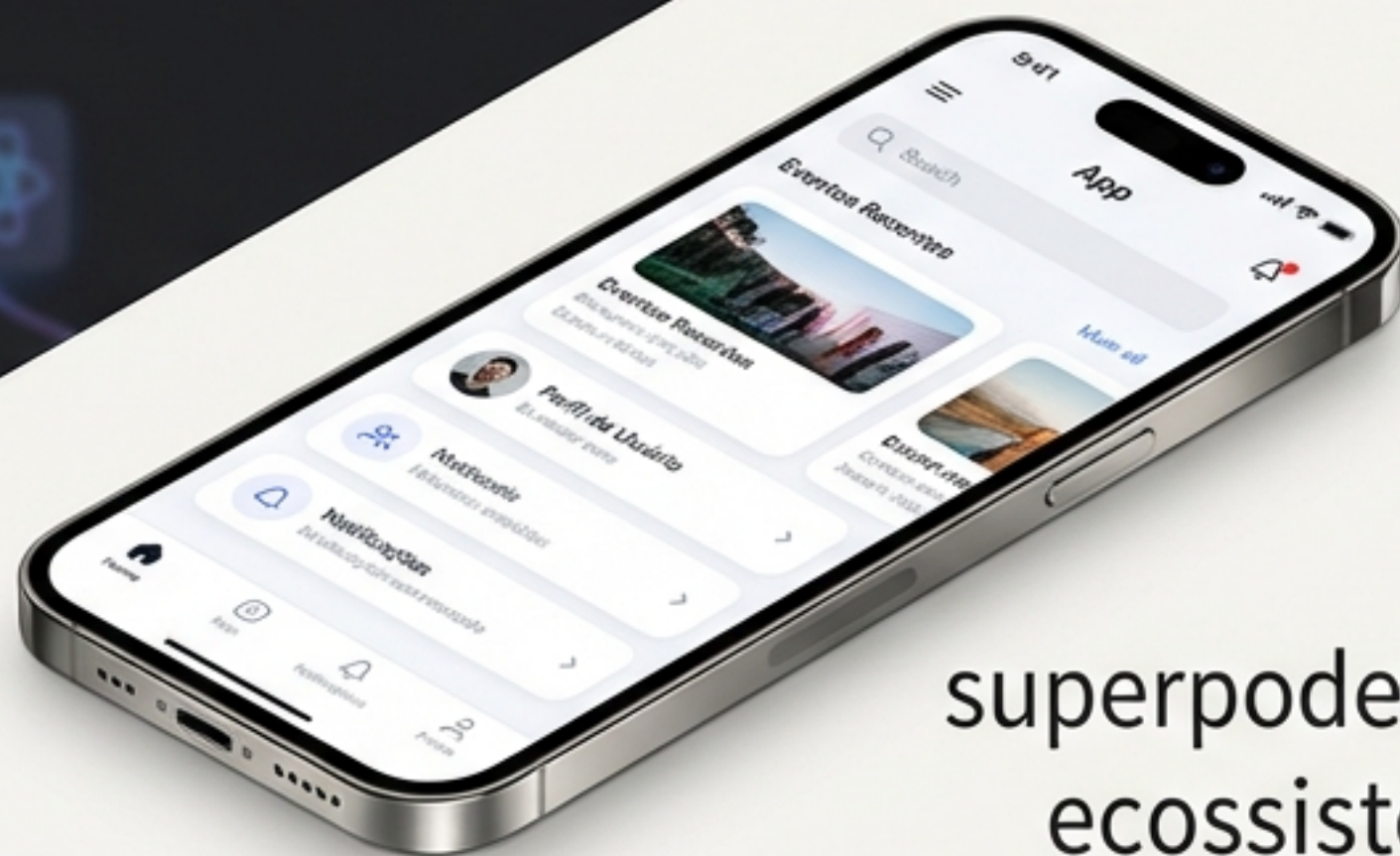


Do Navegador para o Bolso:

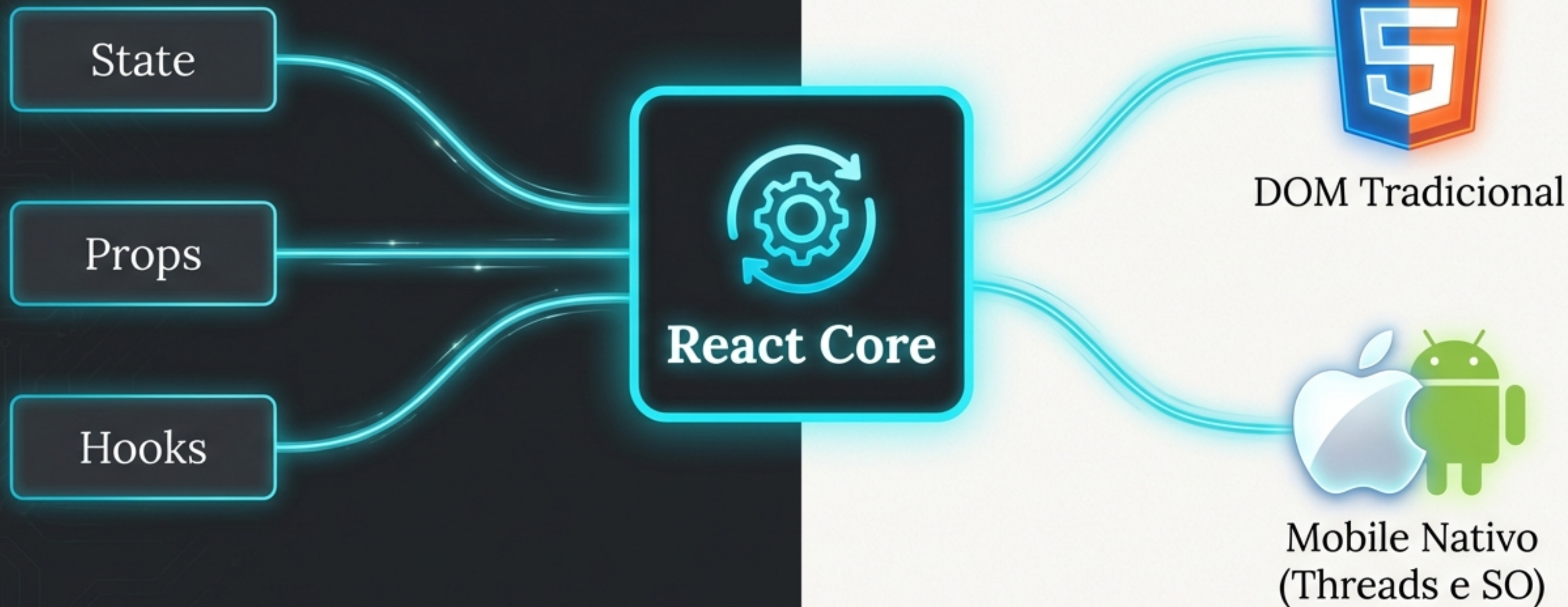
React Native para Desenvolvedores Web

```
export func App() {  
  const (i) => [  
    return  
    ReactNative para desenvolvimento  
  </ReactNative>  
  </>  
};  
const component =  
  render(  
  ));
```



Traduzindo seus
superpoderes de React para o
ecossistema mobile nativo.

A Vantagem Injusta



Panel 1

O React dita a lógica (Unidirectional data flow, Component composition, Hooks).

Panel 2

O React Native apenas altera as ferramentas de pintura.

Panel 3

Não há mais manipulação de DOM tradicional, e sim comunicação via Threads Nativas com o sistema operacional.

O Vocabulário Básico: A Pedra de Roseta

Web (DOM)

Mobile (Native)

`<div>`

(Bloco fundamental de construção)

`<View>`

`<span>` / `<p>`

(Qualquer texto deve estar aqui)

`<Text>`

`<button>`

(Interação tátil)

`<TouchableOpacity>` / `<Button>`

`<img>`

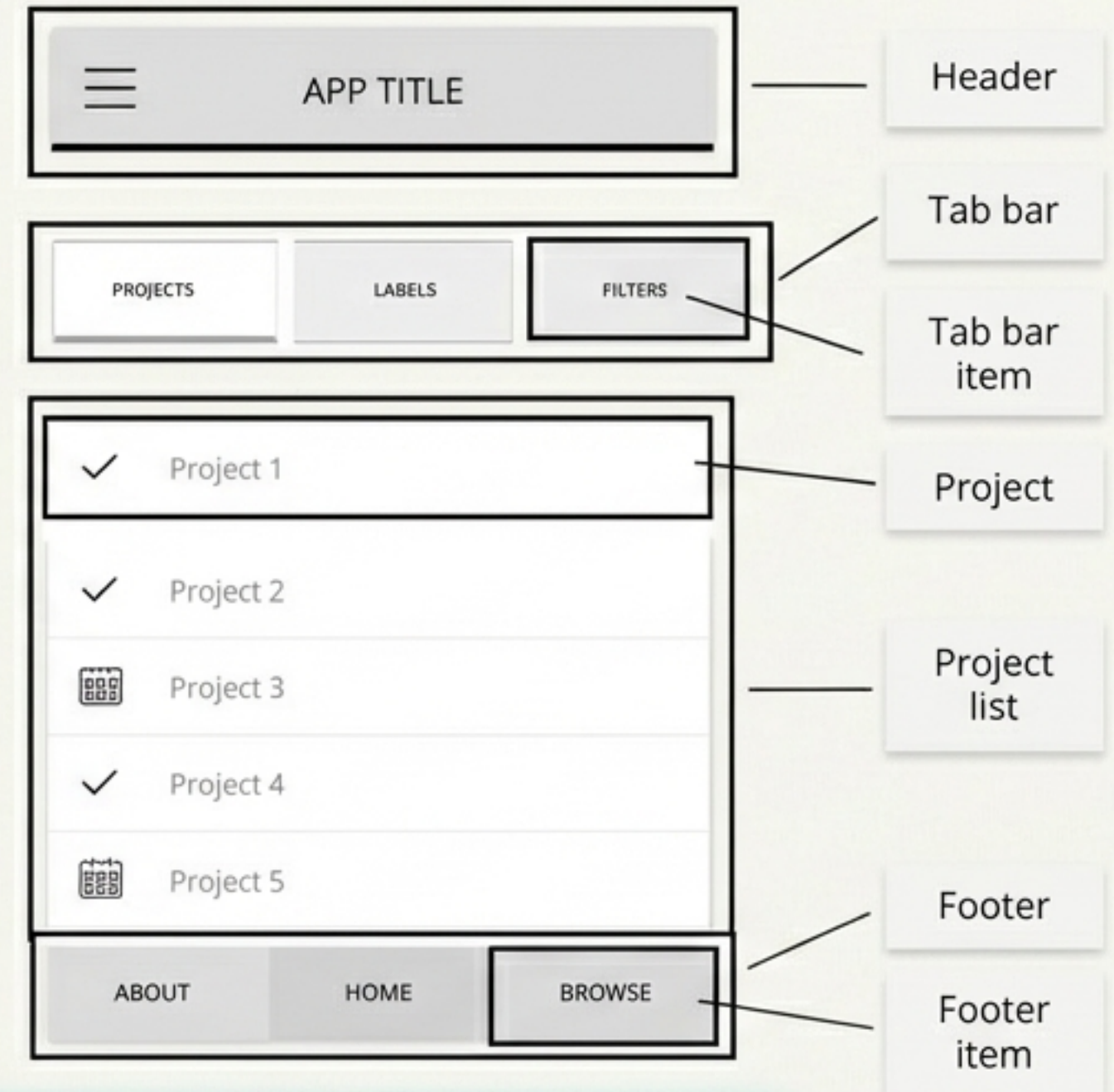
(Renderização visual)

`<Image>`

Importante: No React Native, strings de texto soltas fora de uma tag `<Text>` farão o aplicativo quebrar.

Pensando em Componentes Nativos

```
<Header />  
<TabBar />  
<ProjectList />  
<Footer />
```



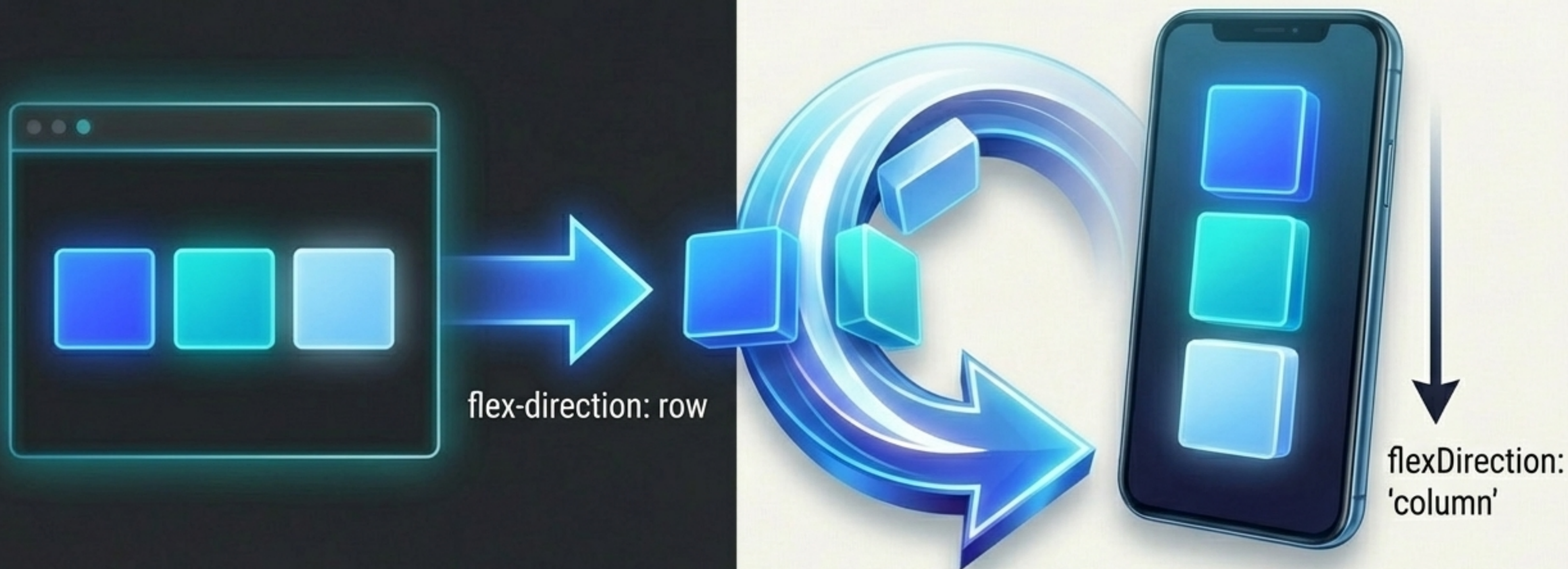
A **decomposição da UI** (Header, Lista de Projetos, Footer) funciona exatamente da mesma maneira que na Web, criando componentes reutilizáveis a partir de `<View>` e `<Text>`.

Estilos como Código: StyleSheet



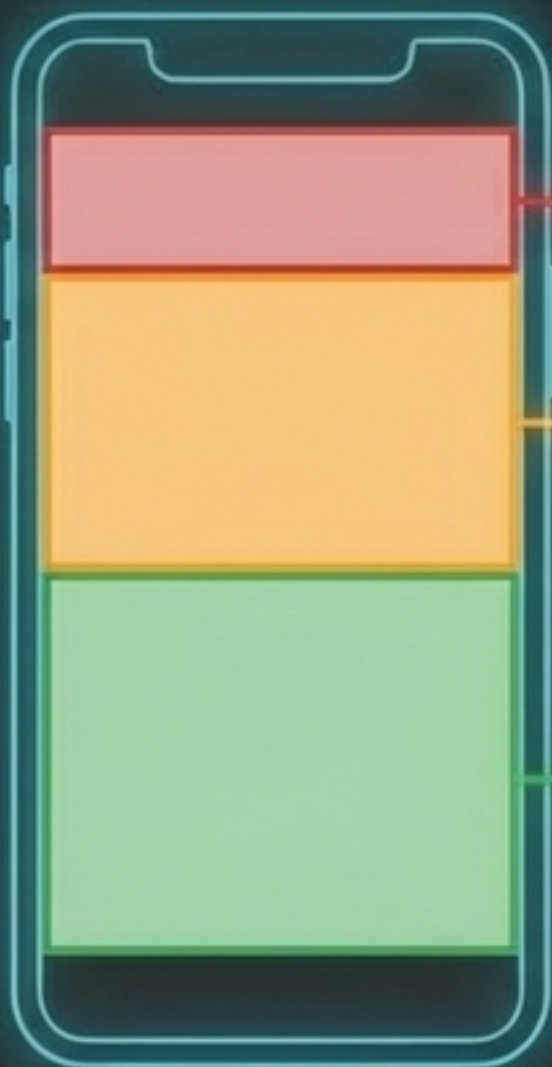
Sem Cascata	camelCase	StyleSheet API	Estilos Dinâmicos
Os estilos são encapsulados localmente, não há efeito de cascata global como na Web.	Adeus hifens. margin-top vira marginTop.	O método StyleSheet.create cria objetos de estilo altamente eficientes e organizados.	Suporte perfeito a arrays para combinar estilos: style={styles.base, styles.active} .

O Motor Yoga: O Novo Padrão é a Coluna



O **React Native** utiliza o **Yoga**, um motor de layout multiplataforma que implementa o Flexbox. A **Grande Diferença**: O valor padrão de **flexDirection** na Web é **row**. No React Native, o padrão é **column**. Seus componentes irão empilhar de cima para baixo automaticamente.

Dominando o Espaço: flex



```
<View style={{ flex: 1, backgroundColor: 'red' }} />
```

```
<View style={{ flex: 2, backgroundColor: 'orange' }} />
```

```
<View style={{ flex: 3, backgroundColor: 'green' }} />
```

- A propriedade **flex** dita como os itens preencherão o espaço disponível ao longo do eixo principal.
- Diferente do CSS web (que exige `flexGrow`, `flexShrink`, `flexBasis`), o Native suporta um **único número inteiro**.
- Matemática de Layout: Se os filhos têm `flex: 1`, `flex: 2` e `flex: 3`, o espaço total é dividido em 6 ($1+2+3$).

Os Controladores de Tráfego: Eixos

Eixo Principal (Main Axis)

justifyContent

- ↑ flex-start (arrow up/start)
- center (centered icon)
- ↓ flex-end (arrow down/end)
- ⋯ space-between (spaced icon)
- ⋯ space-around (around icon)

Eixo Cruzado (Cross Axis)

alignItems

alignItems

- ← flex-start (arrow left/start)
- ⋯ center (centered icon)
- flex-end (arrow right/end)
- ↕ stretch (stretched icon)

Nota de Ouro: Quer centralizar algo perfeitamente em 2D na tela?
`justifyContent: 'center'`
`+ alignItems: 'center'`

O Problema do Gap em Dispositivos Móveis



```
style: {  
gap: 16,  
};
```

Embora versões recentes do Yoga suportem a propriedade CSS gap, o comportamento ainda pode ser inconsistente em diferentes versões de SO (Android/iOS).

```
<Card />  
<Spacer size={16} />  
<Card />
```

A **Solução Real**: Volte para as abordagens à prova de falhas:

- Uso de `marginBottom` nos filhos.
- Criação de um componente `<Spacer size={8} />` reutilizável para separar elementos de forma declarativa e previsível.

Evitando Itens Cortados: flexWrap

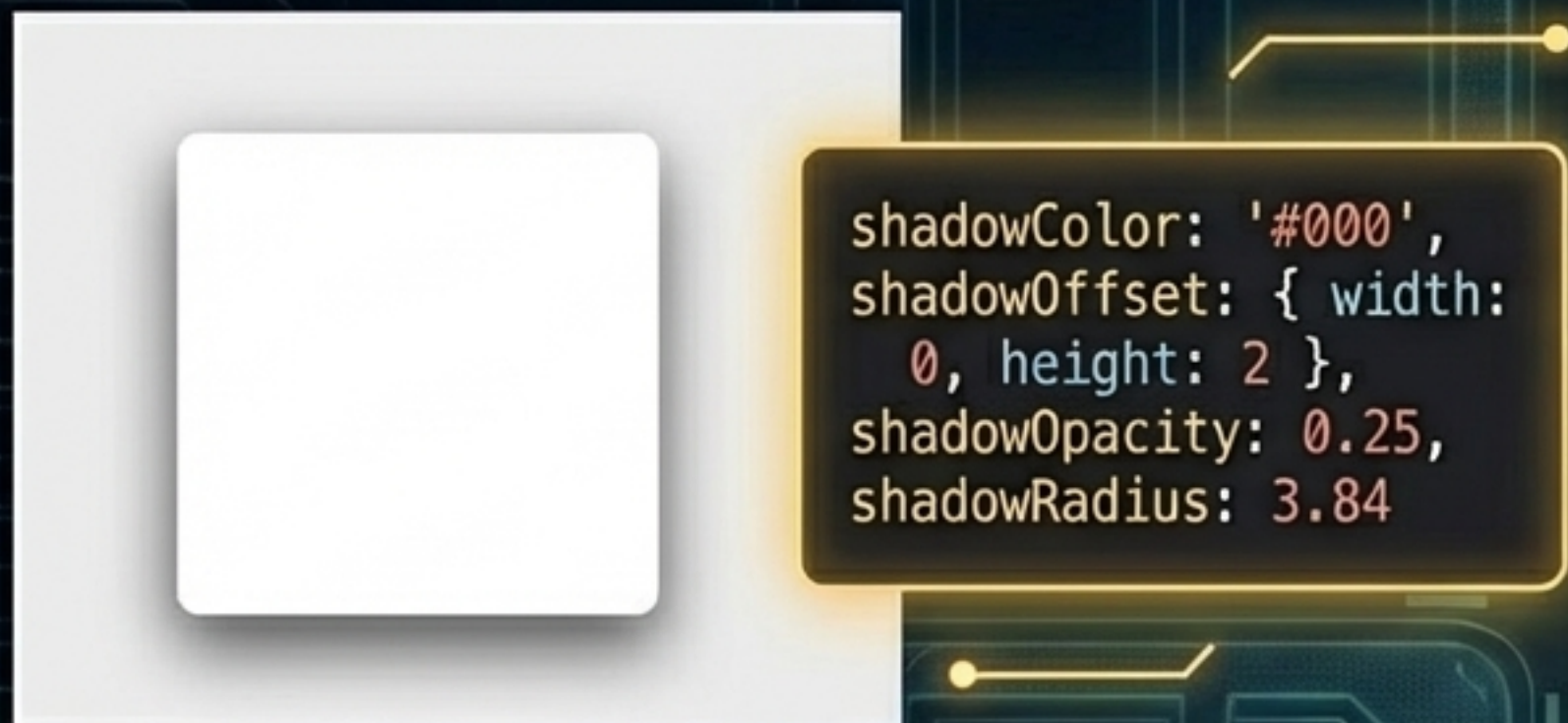


No mobile, os ecrãs são estreitos.

- O padrão do flexbox é amontoar elementos em uma única linha.
- Para elementos empilháveis horizontalmente (como tags ou chips), sempre declare explicitamente `flexDirection: 'row'` e `flexWrap: 'wrap'`.
(Nota: **wrap-reverse** não é suportado no React Native).

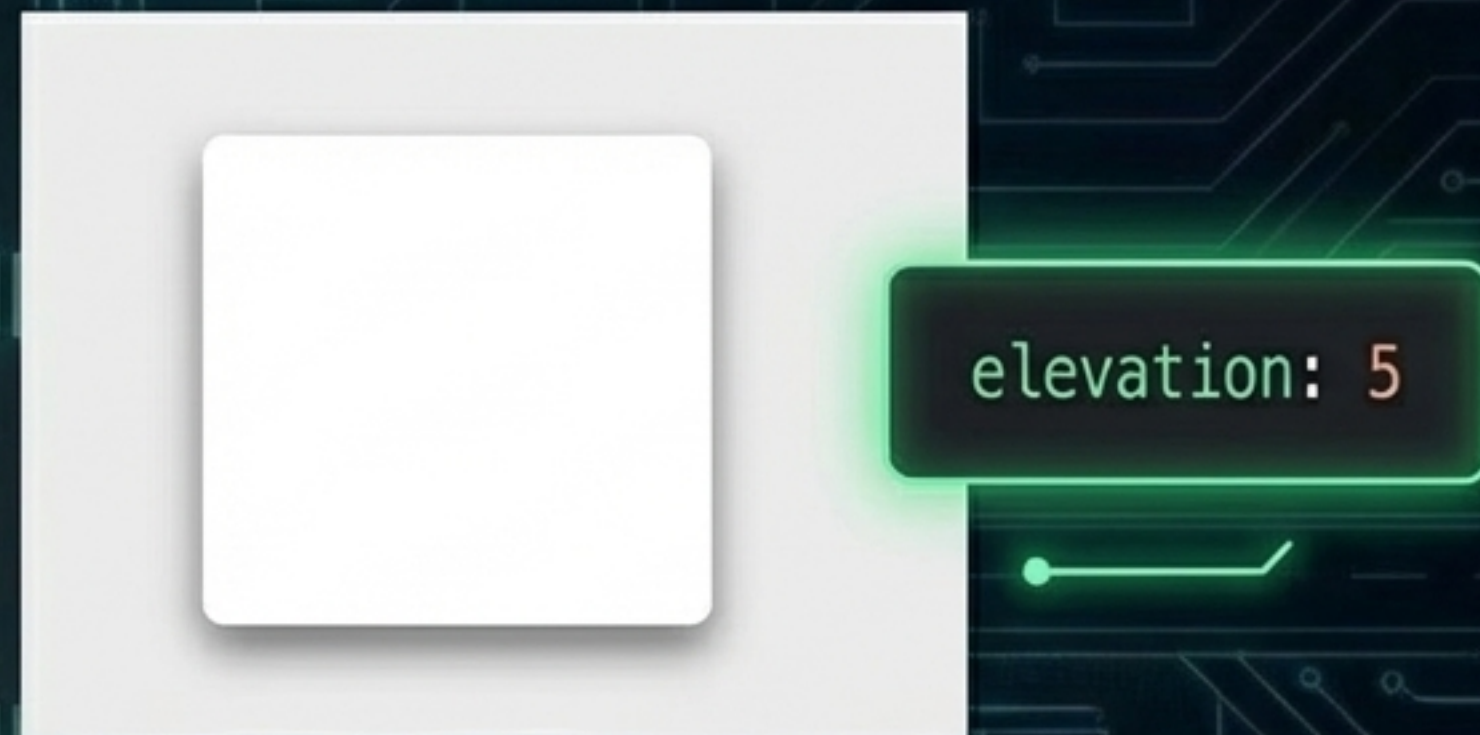
A Batalha Multiplataforma: Sombras

iOS



iOS: Fornece controle granular de sombras em 4 propriedades espelhadas do CSS web.

Android



Android: O motor ignora as propriedades web. Ele utiliza a API exclusiva **elevation**, que desenha sombras nativas baseadas no eixo Z tridimensional do Material Design.

Unificando a Experiência com Platform.OS



```
const styles = StyleSheet.create({
  card: {
    ...Platform.select({
      ios: {
        shadowColor: '#000', shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.2,
      },
      android: {
        elevation: 4,
      },
    }),
  },
});
```



Um único código para dois sistemas operacionais exige ajustes cirúrgicos. O utilitário **Platform** detecta o ambiente em tempo de execução.

Aplica estilos, fontes nativas específicas ou componentes totalmente diferentes usando **Platform.select()**. Isso garante uma sensação 100% nativa sem duplicar código.

Responsividade sem Media Queries

~~@media (min-width: 768px)~~



Não há navegador, logo, não há CSS @media queries.

As dimensões são operadas em pixels independentes de densidade (dp).

A **Solução**: Para reatividade real (orientação retrato vs paisagem), use o hook `useWindowDimensions()`.
Permite renderizações condicionais matemáticas: `width > 600 ? ComponenteTablet : ComponenteCelular`.

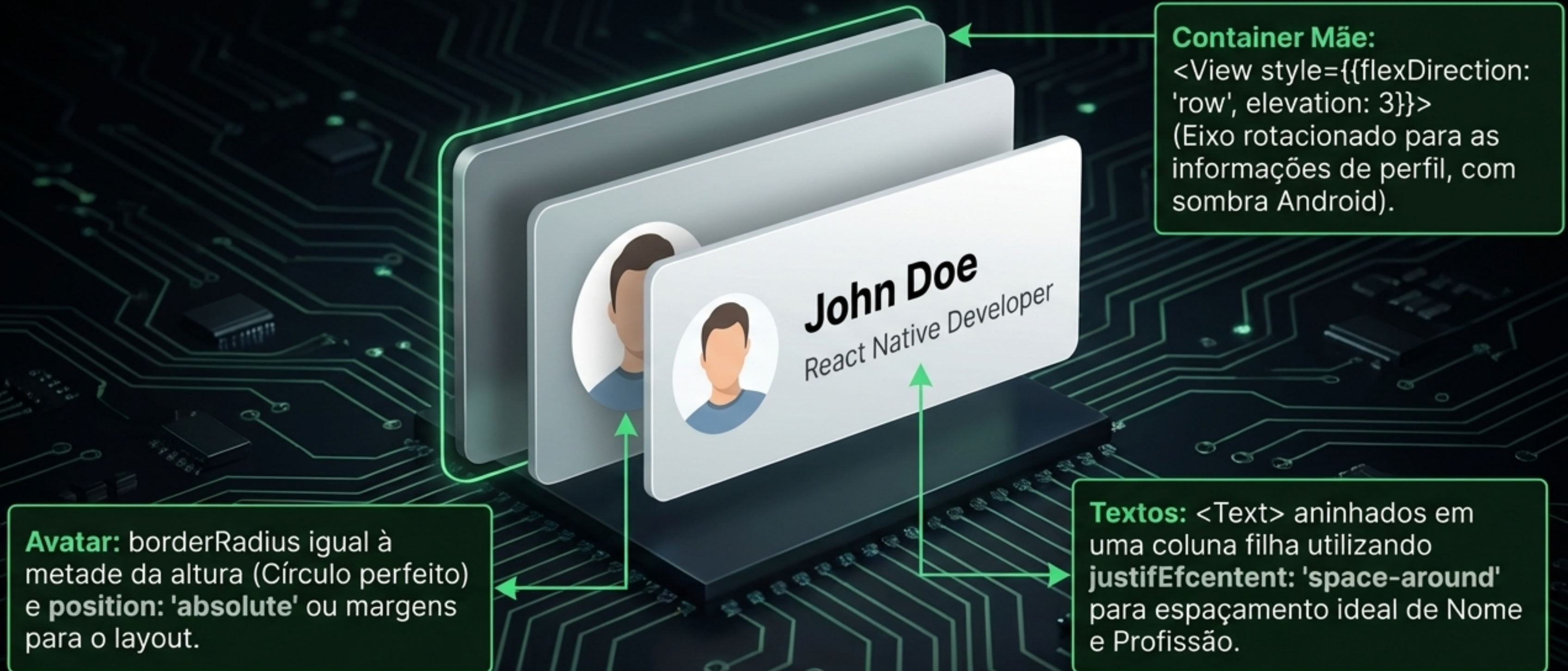
Zonas de Perigo: SafeAreaView

Zona de Perigo



- O hardware moderno invadiu a tela do software.
- Se você posicionar um botão no topo absoluto (top: 0), ele ficará oculto atrás da câmera ou do relógio do sistema no iOS.
- Envolver seu layout com o componente `<SafeAreaView>` injeta automaticamente o padding matemático necessário para respeitar os limites seguros de cada dispositivo.

Síntese: A Anatomia de um Componente Perfeito



O Mesmo Idioma, Um Novo Sotaque

O poder de criar experiências mobile de alta performance,
sem ter que aprender Swift ou Kotlin do zero.

Lembre-se: O React Native abstrai a plataforma, não a performance.

Vocês já conhecem o fluxo de dados; agora, estão equipados
para dominar a tela de forma impecável.

Construam bonito, construam flexível.